

Assignment 7: Proximity Search API

Nearest locations by road distance within a circular radius

Rahul Raj (12341680)
Computer Systems Design Lab

October 9, 2026

API	http://10.1.75.53:3270/search/?lat=&long=&cat=&rad=
Demo page	http://10.1.75.53:3270/
Code	Lab-Task/7.ProximitySearchAPI in the CSD-Course repository

Summary

The API takes a position, a category and a radius, and returns the IDs of the 10 locations of that category that are closest *by road*, out of those lying inside the radius. The 10,000 locations form a 100×100 grid of intersections, and `link.txt` says which neighbouring intersections are connected by a road. About a quarter of the possible roads are missing, so a place that looks close on the map can be a long drive away.

I treated it the way a ride-sharing app treats finding a nearby driver. The rider's position is snapped to the nearest intersection, candidates are restricted to a radius, and they are ranked by how far you actually have to drive. The road distances are computed with a breadth-first search (BFS) on the road graph that stops as soon as the answer is known.

- The results are the same as a brute-force Dijkstra search on 1,600 test queries (21 automated tests).
- In a simulation of the marking scheme on 2,000 random queries, ranking by road distance scores 100/100, while Euclidean and Manhattan ranking would score about 89.6 and 89.0.
- A query takes about 0.025 ms inside the server and explores around 116 intersections. On a synthetic city with a million locations it still takes under 0.5 ms.

1 Problem

The inputs are `lat`, `long`, `cat` and `rad`, and the output is a list of 10 location IDs. The assignment uses two different distances:

- **Radius (straight line).** A location can only be returned if its Euclidean distance from `(lat, long)` is at most `rad`.
- **Closeness (along the grid).** Among those locations, “closest” means the shortest distance travelled along the roads. Roads only join neighbouring grid points (no diagonals), and some of them are missing.

Two consequences matter for the algorithm. A location inside the circle can be far away by road, and the shortest road to a location inside the circle can go outside the circle and come back in.

2 Data

2.1 Locations and roads

Property	Value
Locations	10,000 on a 100×100 grid, spacing 1/99
Categories	8 categories, 1,250 locations each
Possible roads between neighbours	19,800
Roads in <code>link.txt</code>	14,800 (74.7%)
Missing roads	5,000 (2,511 east–west, 2,489 north–south)
Connected components	1
Roads per intersection	1: 111, 2: 2,601, 3: 4,865, 4: 2,423
Independent loops (roads – nodes + 1)	4,801

Table 1: The dataset and the road network (from `scripts/analyze.py`).

The locations form an exact grid. Location IDs follow the rule $ID = 100 \cdot \text{row} + \text{col} + 1$, where $\text{row} = 99 \cdot \text{lat}$ and $\text{col} = 99 \cdot \text{long}$. So I can turn coordinates into an array index directly, with no lookup table.

Each line of `link.txt` has the form `long_A lat_A long_B lat_B`. All 14,800 lines join two neighbouring points, no line is repeated, and in every line point A has the smaller coordinate. So each road is written once in a fixed order, which means roads can be used in both directions. I also tried reading the lines as one-way roads. Then most of the map becomes unreachable (from node 6890 only 112 nodes can be reached), so that reading is not what was intended (see also Section 3.3). As an undirected graph the network is connected. It is a spanning tree (9,999 roads) plus 4,801 extra roads that form loops, and it has 111 dead ends.

2.2 How much the missing roads matter

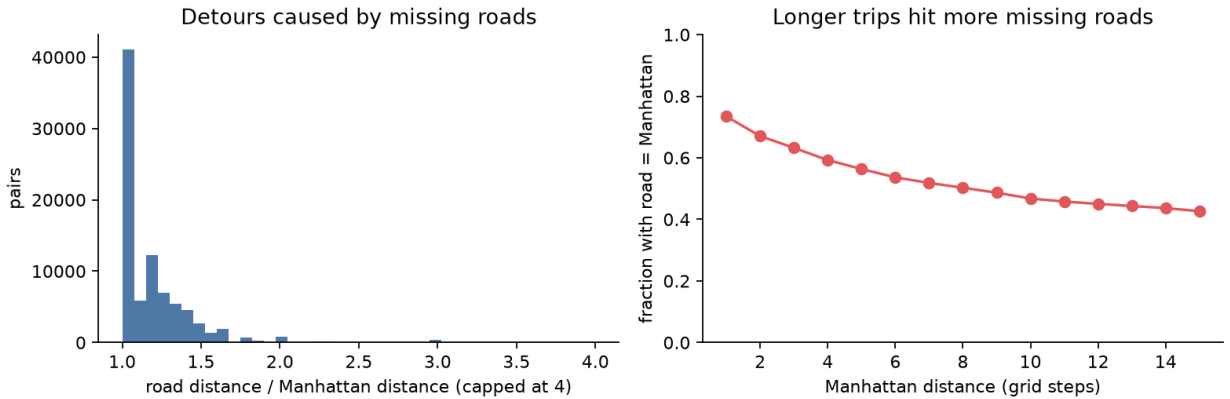


Figure 1: Left: road distance divided by Manhattan distance for all pairs at most 15 steps apart, from 200 random starting points. Right: fraction of pairs where the road distance equals the Manhattan distance.

Pairs	Road = Manhattan	Mean ratio	90th pct	99th pct	Max
84,871	48.4%	1.18	1.46	2.14	13.0

Table 2: Detour ratio (road distance / Manhattan distance).

Only about half of the short trips can be driven along a shortest grid path. Longer trips are more likely to run into a missing road, and the worst detour is 13 times the Manhattan distance.

3 Choosing the distance

3.1 Options I considered

Distance	Models	Decision
Euclidean	flying in a straight line	used for the radius only; it ignores the roads
Manhattan	driving on a complete grid	only correct when no road on the way is missing
Chebyshev / 8 neighbours	diagonal moves	the assignment says there are no diagonal links
Haversine	distance on the Earth	the coordinates are a unit square, not degrees
Shortest path on the road graph	driving on the actual roads	used for ranking
Shortest path, one-way roads	directed streets	does not fit the data (Section 3.3)
Travel time with turns or traffic	what real ride-sharing apps use	there is no speed, turn or traffic data

Table 3: Candidate distance measures.

Every road segment has the same length (1/99), so the shortest drive is the path with the fewest segments. That is exactly what BFS finds, and Dijkstra’s priority queue is not needed.

3.2 Comparison with ride sharing

A ride-sharing app matching a rider to a driver does roughly the same steps:

1. **Map matching:** the rider’s GPS position is snapped to the road network. Here, the query point is snapped to the nearest intersection.
2. **Candidates by area:** drivers within some radius are taken from a spatial index. Here, the rad filter.
3. **Ranking by driving distance:** candidates are ordered by route length or ETA, not by straight-line distance, since a driver on the other side of a river is not really close. Here, the BFS distance.

3.3 What each distance would score

I generated 2,000 random queries: half placed on a location and half anywhere, with radius between 0.08 and 0.35, and only queries with at least 10 valid answers were kept. For each one I computed the correct answer with a separate Dijkstra search and scored every ranking the way the assignment does: one point for each returned ID that is correct. IDs tied with the correct 10th place also count, since the TA said any tied location is accepted.

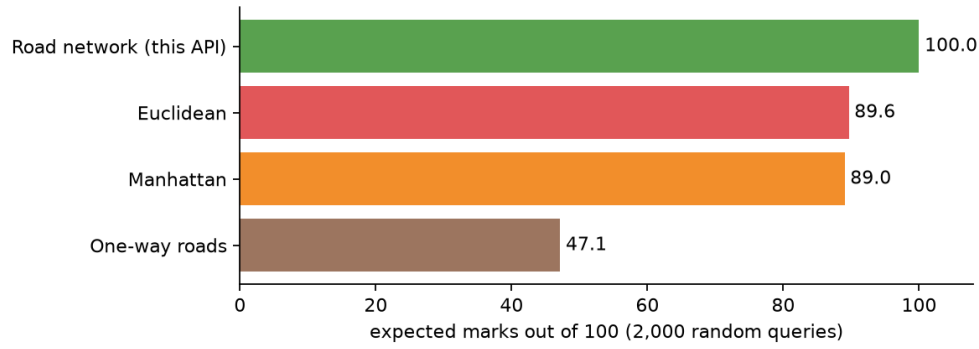


Figure 2: Expected marks out of 100 for each ranking.

Ranking	Expected score / 100	Queries fully correct	Worst query
Road network (this API)	100.0	100.0%	10/10
Euclidean	89.6	30.9%	4/10
Manhattan	89.0	28.9%	3/10
One-way roads	47.1	4.0%	0/10

Table 4: Simulated marks for 2,000 random queries.

Euclidean and Manhattan get about 9 out of 10 IDs right on average, so at first they seem fine. But they get the whole list right in less than a third of the queries, and in some places they miss most of it. The one-way reading scores under half, which agrees with the roads being two-way.

3.4 Example

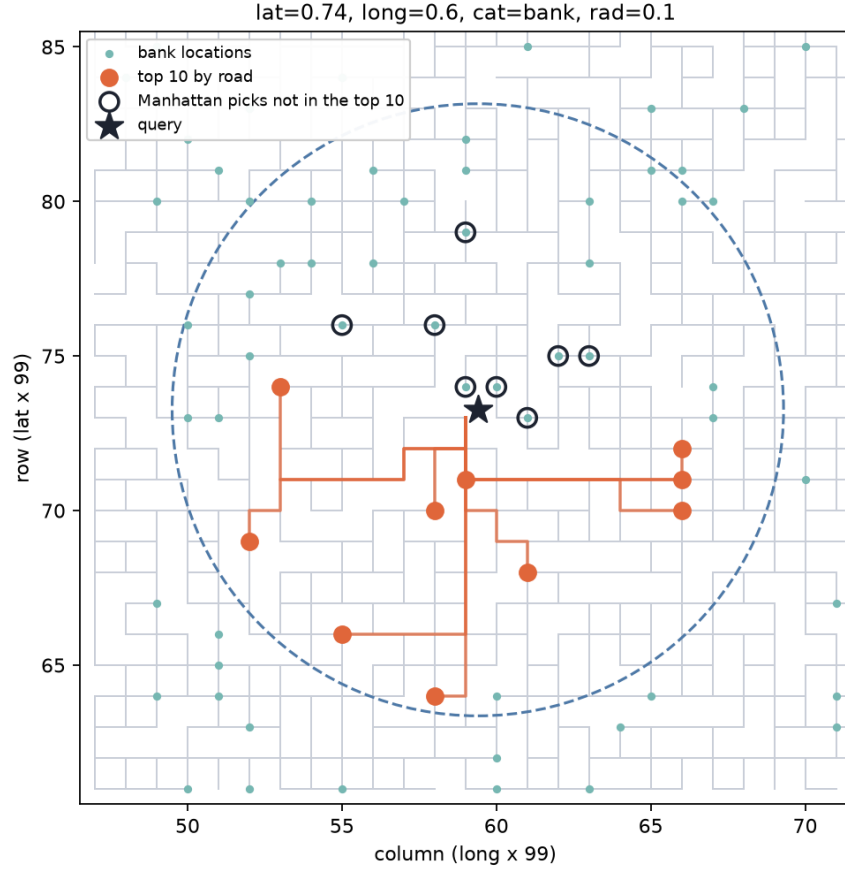


Figure 3: Query $\text{lat}=0.74$, $\text{long}=0.6$, $\text{cat}=\text{bank}$, $\text{rad}=0.1$. Grey lines are roads (gaps are missing roads). Orange shows the 10 results and the routes to them. Circled banks are the ones Manhattan ranking would return instead.

Rank	ID	(lat, long)	Road steps	Manhattan steps	Straight line (steps)
1	7160	(0.7172, 0.5960)	2	2	2.30
2	7059	(0.7071, 0.5859)	4	4	3.55
3	6862	(0.6869, 0.6162)	7	7	5.50
4	7167	(0.7172, 0.6667)	9	9	6.98
5	7267	(0.7273, 0.6667)	10	8	6.72
6	7067	(0.7071, 0.6667)	10	10	7.36
7	6459	(0.6465, 0.5859)	10	10	9.37
8	7454	(0.7475, 0.5354)	11	7	6.44
9	6656	(0.6667, 0.5556)	11	11	8.49
10	6953	(0.6970, 0.5253)	11	11	8.54

Table 5: Results for the example query.

The intersection nearest to the query has only one road, which goes south. Eight banks just north and east of the query are 1 to 5 steps away in Manhattan distance, but they are far by road, so the road ranking picks banks to the south and east instead. Manhattan ranking would get only 2 of the 10 right here.

4 Algorithm

4.1 Data structures

All of this is built once when the server starts, which takes about 25 ms.

- `adj[v]`: the neighbours of each node (14,800 roads, so 29,600 entries). The node number is $100 \cdot \text{row} + \text{col}$, so a coordinate gives its node directly.
- `ids`, `lat`, `lon`, `cat`: plain lists indexed by node.
- For each category, numpy arrays with the coordinates of its 1,250 locations. These make it quick to count how many of them are inside the circle.

4.2 BFS with early stopping

```
55 def search(self, lat, lon, cat, rad, k=10, want_paths=False):
56     start_time = time.perf_counter()
57     net = self.net
58
59     # If fewer than k places are inside the circle we can stop once we have
60     # found all of them, instead of searching the whole map.
61     need = min(k, self.count_in_radius(lat, lon, cat, rad))
62
63     src = net.node_of(lat, lon)
64     dist = {src: 0}
65     parent = {src: -1}
66     queue = deque([src])
67     hits = []
68     cutoff = None # road distance of the need-th hit
69
70     while queue and need > 0:
71         v = queue.popleft()
72         d = dist[v]
73         # BFS pops nodes in order of distance, so once we are past the
74         # distance of the need-th hit nothing closer can turn up. Nodes at
75         # exactly that distance are still collected (ties).
76         if cutoff is not None and d > cutoff:
77             break
78
79         if net.cat[v] == cat:
80             e = math.hypot(net.lat[v] - lat, net.lon[v] - lon)
81             if e <= rad + EPS:
82                 hits.append(Hit(v, net.ids[v], d, e))
83                 if len(hits) == need:
84                     cutoff = d
85
86         # Expand even outside the circle: the shortest road to a place
87         # inside may go out and come back in.
88         for w in net.adj[v]:
89             if w not in dist:
90                 dist[w] = d + 1
91                 parent[w] = v
92                 queue.append(w)
93
94         # closest by road first, then by straight line, then lower ID
95         hits.sort(key=lambda h: (h.hops, h.euclid, h.id))
96         elapsed = (time.perf_counter() - start_time) * 1000
97         return SearchResult(src, hits[:k], len(dist), need, elapsed, parent if want_paths else
98                             None)
```

The search works like this:

1. Snap the query to its nearest node s and count $T = \min(10, \text{locations inside the circle})$.

2. Run BFS from s . When a node of the right category inside the circle is reached, add it to the results.
3. When the T -th result is found, remember its distance d_T . Keep going until BFS reaches a node farther than d_T , then stop.
4. Sort the results by (road distance, straight-line distance, ID) and return the first 10.

Why stopping early is correct. BFS takes nodes out of the queue in order of distance. Once it reaches a node farther than d_T , every node not yet seen is at least that far, so none of them can be in the top 10. Stopping after the whole d_T level, not right at the 10th result, keeps every location tied with the 10th, and the tie-break then chooses between them.

Why the circle does not limit the search. The shortest road to a location inside the circle can leave the circle, so nodes outside it still have to be expanded. The circle only decides which nodes count as results. T handles the case with fewer than 10 locations in the circle: once all of them are found the search stops, instead of going through the whole map looking for ones that do not exist.

Unreachable locations. Locations that cannot be reached are never returned, because BFS never gets to them. The given network is connected, but the code does not depend on that.

4.3 Complexity

Step	Cost
Count locations in the circle	$O(n/8)$ with numpy (1,250 points)
BFS	$O(V' + E')$ for the explored part only, $V' \approx 116$ on average
Sort results	$O(h \log h)$, $h \approx 10\text{--}15$
Memory	$O(V + E)$, a few MB

Alternatives.

- **Full Dijkstra for every query** (used as the reference in the tests) is about 130 times slower.
- **Precomputing all distances** ($10^4 \times 10^4$ values, about 200 MB) is not needed at this speed.
- **A* to each candidate** would repeat work that a single BFS shares.
- **A k-d tree** would only speed up the radius count, which already takes microseconds.

For a large real road network with different road lengths, I would switch to Dijkstra or A*, or to contraction hierarchies.

4.4 Edge cases

Case	What the API does	Why
Query between intersections	snap to the nearest intersection (round each coordinate); points outside the square go to the border	a car starts from the nearest intersection; for queries on a grid point this is exact
Location exactly on the circle	counted as inside, with a tolerance of 10^{-6}	the CSV rounds to 6 decimals, so a point exactly 3 steps away comes out about $5 \cdot 10^{-7}$ outside a radius of 3/99
Equal road distances	then straight-line distance, then ID	any tie is accepted; a fixed order makes results repeatable
Category text	spaces trimmed, case ignored	Hospital and hospital are the same
Fewer than 10 valid	return the ones that exist	never return invalid IDs

5 API

Route	Description
GET /search/?lat=&long=&cat=&rad=	returns {"ids": [...] } with 10 IDs, closest by road first
GET /search?...&debug=1	same handler; registered separately so there is no 307 redirect also returns road steps, straight-line and Manhattan distance, detour ratio and the full route for each result
GET /health	number of locations and roads, categories, start-up time
GET /	demo page with a map (Figure 4); GET /docs is the FastAPI docs page

Bad input returns HTTP 400 with a message naming the field: a missing parameter, a value that is not a number, an unknown category, or a negative radius. CORS is enabled for GET requests.

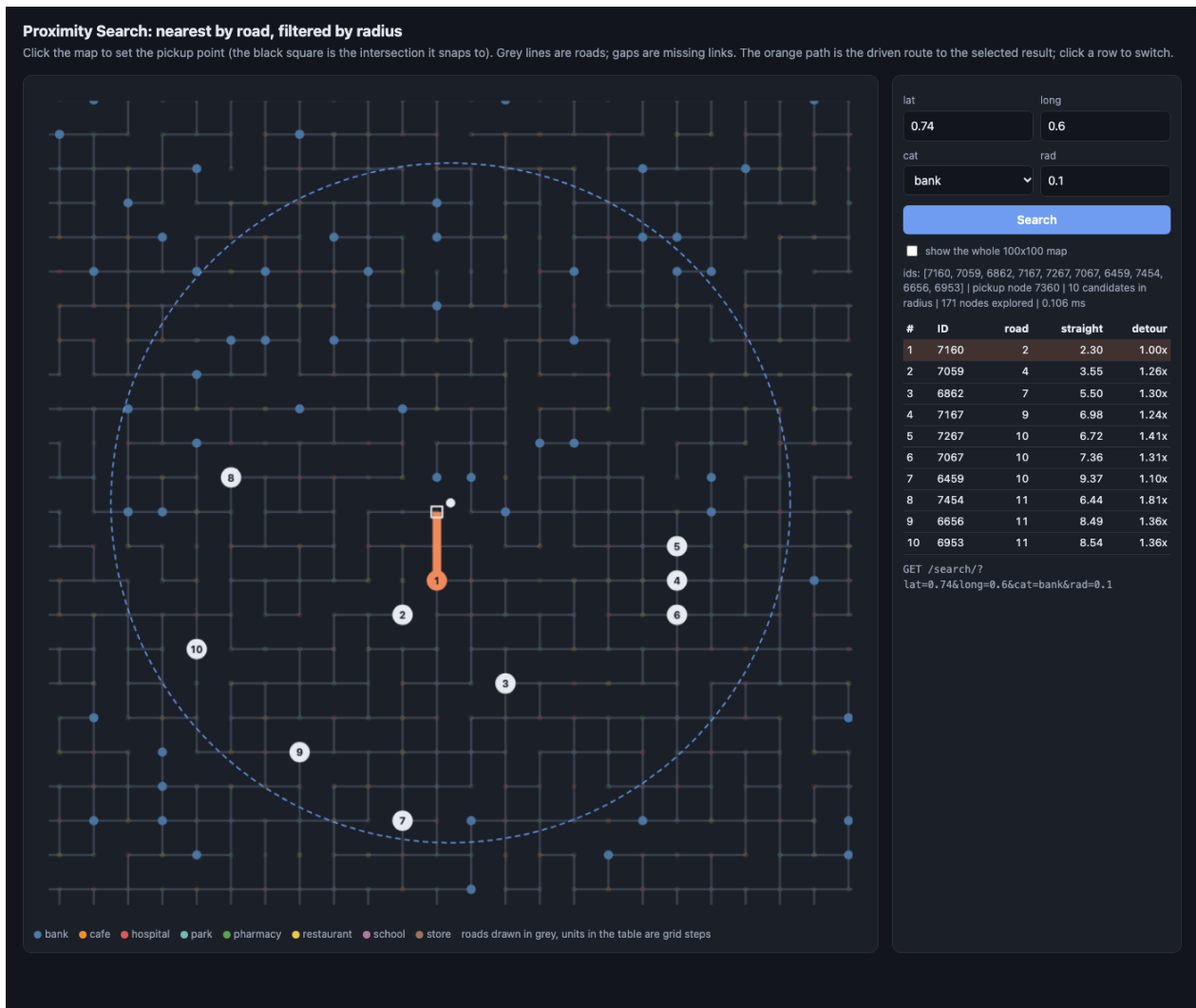


Figure 4: The demo page for the same query as the example above. The square marks the intersection the query snaps to, and the orange line is the route to result 1.

6 Testing

- **Against brute force:** for 1,600 random queries (half on grid points, radius from 0.02 to 2, all categories) the BFS gives exactly the same list, in the same order, as Dijkstra to all 10,000 nodes followed by a sort.
- **Small hand-made networks:** a 3×3 grid with one missing road checks the exact order of the detours and the tie-break. A grid where some places cannot be reached checks that they are not returned. A diagonal link in the input is rejected.
- **Edge cases:** a point exactly on the circle, the corner of the map, points outside the map, always 10 results when 10 exist, every result inside the circle and of the right category, and Manhattan distances when there is no link file.
- **API:** route and parameter names, response format, no redirect without the slash, category case, the routes in debug output, and 400 for bad input.

01_tests.txt

```
$ .venv/bin/python -m pytest -v
tests/test_api.py::test_search_returns_ten_ids PASSED
tests/test_api.py::test_search_without_slash PASSED
```

```

tests/test_api.py::test_category_case_and_spaces PASSED
tests/test_api.py::test_debug_output PASSED
tests/test_api.py::test_bad_input_gives_400 PASSED
tests/test_api.py::test_health_and_demo_page PASSED
tests/test_search.py::test_dataset PASSED
tests/test_search.py::test_every_location_is_reachable PASSED
tests/test_search.py::test_bfs_matches_brute_force[0] PASSED
tests/test_search.py::test_bfs_matches_brute_force[1] PASSED
tests/test_search.py::test_bfs_matches_brute_force[2] PASSED
tests/test_search.py::test_bfs_matches_brute_force[3] PASSED
tests/test_search.py::test_returns_ten_when_ten_exist PASSED
tests/test_search.py::test_results_are_valid_and_sorted PASSED
tests/test_search.py::test_point_on_the_circle_is_included PASSED
tests/test_search.py::test_corner_and_outside_points PASSED
tests/test_search.py::test_road_ranking_differs_from_manhattan PASSED
tests/test_search.py::test_missing_road_forces_a_detour PASSED
tests/test_search.py::test_unreachable_places_are_skipped PASSED
tests/test_search.py::test_diagonal_link_is_rejected PASSED
tests/test_search.py::test_without_link_file_distance_is_manhattan PASSED
===== 21 passed in 5.60s =====

```

02_curl_public.txt

```

$ curl -s 'http://10.1.75.53:3270/health'
{"status":"ok","locations":10000,"grid":"100x100","roads":14800,"roads_missing":5000,"road_file":"link.txt",
↪ "categories":["bank","cafe","hospital","park","pharmacy","restaurant","school","store"],"load_ms":114.6}
↪ [HTTP 200]

$ curl -s 'http://10.1.75.53:3270/search/?lat=0.74&long=0.6&cat=bank&rad=0.1'
{"ids": [7160,7059,6862,7167,7267,7067,6459,7454,6656,6953]} [HTTP 200]

$ curl -s 'http://10.1.75.53:3270/search?lat=0.5&long=0.5&cat=cafe&rad=0.12'
{"ids": [5051,5053,5251,4949,5154,5149,4751,5047,4748,4653]} [HTTP 200]

$ curl -s 'http://10.1.75.53:3270/search/?lat=0.2&long=0.9&cat=Hospital&rad=0.2'
{"ids": [2290,2389,2490,2388,2491,2194,1786,2285,1688,2195]} [HTTP 200]

$ curl -s 'http://10.1.75.53:3270/search/?lat=0.5&long=0.5&cat=spaceport&rad=0.1'
{"error":"unknown category 'spaceport'; expected one of ['bank', 'cafe', 'hospital', 'park', 'pharmacy',
↪ 'restaurant', 'school', 'store']"} [HTTP 400]

$ curl -s 'http://10.1.75.53:3270/search/?lat=0.5&long=abc&cat=cafe&rad=0.1'
{"error":"'long' must be a number, got 'abc'"} [HTTP 400]

$ curl -s 'http://10.1.75.53:3270/search/?lat=0.5&long=0.5&cat=cafe'
{"error":"missing required query parameter 'rad'"} [HTTP 400]

```

7 Performance

7.1 Search time

Method (10,000 locations)	p50 (ms)	p95 (ms)	p99 (ms)	Nodes explored
BFS with early stop (API)	0.025	0.035	0.042	116
Dijkstra to all nodes + scan	3.27	3.81	4.04	10,000

Table 6: Time per query inside the server, 5,000 random queries.

7.2 Larger cities

To check that the search time does not grow with the size of the map, I generated random cities the same way `link.txt` looks: a random spanning tree plus extra roads up to 75% of all possible roads, with random categories. The radius was 12 grid steps every time.

Locations	Roads	Load (s)	p50 (ms)	p99 (ms)	Nodes explored
10,000	14,850	0.0	0.023	0.037	115
99,856	149,310	0.2	0.055	0.083	118
1,000,000	1,498,500	3.6	0.286	0.414	118

Table 7: Search time on synthetic cities.

BFS explores roughly the same number of nodes at every size, because it only goes as far as the 10 closest results. The small increase at one million comes from counting the locations in the circle, which is linear in the number of locations of the category.

7.3 Over HTTP

03_loadtest_local.txt (laptop)

```
$ scripts/loadtest.py http://127.0.0.1:8077 --requests 2000 --concurrency 1
requests=2000 concurrency=1 errors=0 wall=1.20s throughput=1663 req/s p50=0.6ms p95=0.7ms p99=0.9ms
↪ mean=0.6ms
$ scripts/loadtest.py http://127.0.0.1:8077 --requests 5000 --concurrency 32
requests=5000 concurrency=32 errors=0 wall=7.99s throughput=626 req/s p50=32.1ms p95=166.2ms p99=290.6ms
↪ mean=51.0ms
```

03_loadtest_public.txt (deployed API)

```
$ scripts/loadtest.py http://10.1.75.53:3270 --requests 2000 --concurrency 1
requests=2000 concurrency=1 errors=0 wall=20.97s throughput=95 req/s p50=8.4ms p95=11.5ms p99=94.5ms
↪ mean=10.5ms
$ scripts/loadtest.py http://10.1.75.53:3270 --requests 5000 --concurrency 32
requests=5000 concurrency=32 errors=0 wall=13.19s throughput=379 req/s p50=42.8ms p95=252.3ms p99=399.3ms
↪ mean=84.2ms
```

On the laptop a single client gets a response in about 0.6 ms. With 32 clients the laptop handles over 600 requests per second; at that point the load generator running on the same machine is the bottleneck. Against the deployed API (one CPU core, through the lab NAT) the median response time with one client is about 8 ms, which is mostly network delay, and 32 clients get around 380 requests per second. There were no errors in any run. The handler is `async` because a search takes much less than a millisecond, so running it in a thread pool would cost more than the search itself.

8 Deployment

The API runs on lab machine `stu78_sys2` in `~/assignment7` on port 3000, which the lab NAT forwards to `http://10.1.75.53:3270`. The lab machines have no internet access, no cron and no tmux, and their Python cannot create a normal virtual environment because `ensurepip` is missing. So `scripts/deploy.sh` does the following:

1. downloads the Linux x86-64 wheels for Python 3.12 on the laptop, using the versions pinned in `requirements.txt`;
2. copies the code, the data and the wheels over SSH with `tar` (there is no rsync on the machines);
3. creates a virtual environment with `--without-pip --system-site-packages`, and installs the wheels offline with the system's pip;
4. starts a small loop script with `setsid nohup`. It restarts uvicorn if it exits, and keeps pid files so that `deploy.sh stop` and `deploy.sh status` work without `pskill`.

04_restart_after_kill.txt

```
$ ssh lbsys2 'cd ~/assignment7; kill -9 $(cat server.pid); sleep 4; cat server.pid; curl  
↳ localhost:3000/health'  
killed 873, now 955  
health 200  
(supervisor loop restarted uvicorn within ~1 s; public URL kept answering after restart)
```

9 Limitations and future work

- **Road lengths and speeds:** if roads had different lengths or speed limits, BFS would be replaced by Dijkstra or A*, and the ranking would become an actual travel time.
- **One-way roads and turns:** the graph can be made directed, and turn costs can be added by splitting each intersection into one node per direction.
- **Traffic and closed roads:** road weights could be updated while the server is running. Since nothing is precomputed, a change would apply from the next query on.
- **Starting point:** instead of snapping to one intersection, the search could start from both ends of the road the rider is on, each with the remaining distance along that road.